

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessor - Microcontroller

Lab Report

Report 1

Lecturer: Lê Văn Sỹ

Student(s): Dương Nhật Khang 2452450

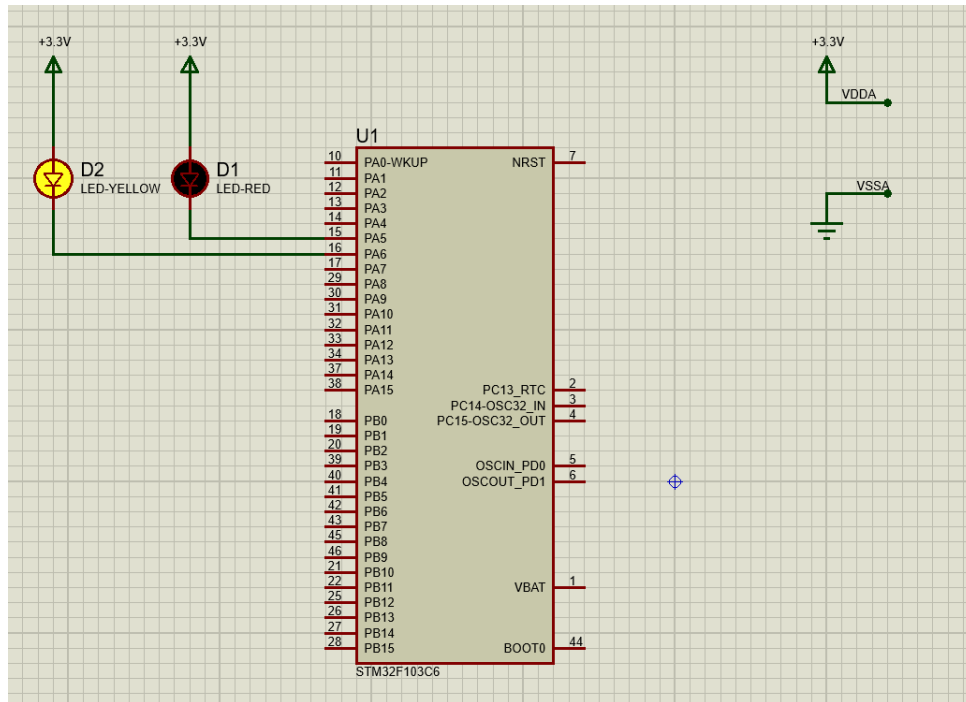
HO CHI MINH CITY, SEPTEMBER 2026

Contents

1 Exercise 1	3
2 Exercise 2	4
3 Exercise 3	6
4 Exercise 4	8
5 Exercise 5	12
6 Exercise 6	15
7 Exercise 7	17
8 Exercise 8	17
9 Exercise 9	17
10 Exercise 10	18
11 References	19

1 Exercise 1

From the simulation on Proteus, one more LED is connected to pin PA6 of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is LED-YELLOW, which can be found from the device list.



<https://github.com/kyoyamakhang/MCU-MPU-Lab-Files-Archive>

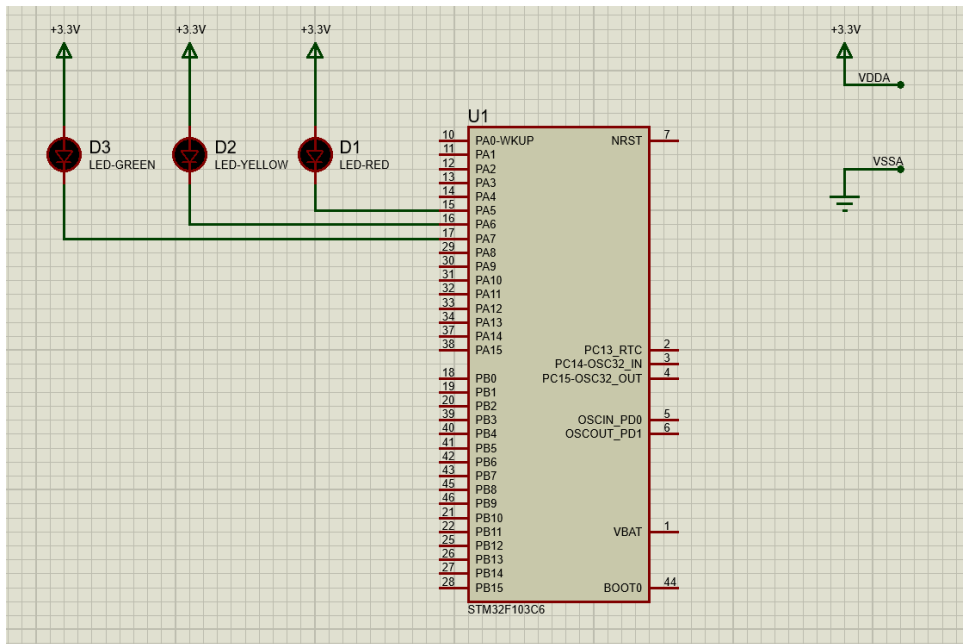
The Schematic shown above contains 2 LEDs, 1 RED and 1 YELLOW, connected to output pin PA5 and PA6 respectively. In this schematic, the LEDs are active-low, meaning they turn on when the output is low, this is due to the source being connected to the positive pin of the diode. The reference voltages can be seen on the top right of the schematic.

```
1 while (1)
2 {
3     HAL_GPIO_TogglePin(GPIOA , GPIO_PIN_5);
4     HAL_GPIO_TogglePin(GPIOA , GPIO_PIN_6);
5     HAL_Delay(2000);
6 }
```

Listing 1: Code inside while loop

2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named LED-GREEN is added to the system, which is connected to PA7. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.



<https://github.com/kyoyamakhang/MCU-MPU-Lab-Files-Archive>

Like the previous exercise, the schematics for this exercise is to be expanded with a GREEN LED which is connected to output pin PA7. Again, this LED is also active low just like RED and YELLOW LEDs.

```
1 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); // RED
2 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // YELLOW
3 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET); // GREEN
4 HAL_Delay(2000); // initial 2s delay for first state - GREEN
5 /* USER CODE END 2 /
6
7 / Infinite loop /
8 / USER CODE BEGIN WHILE /
9 while (1)
```



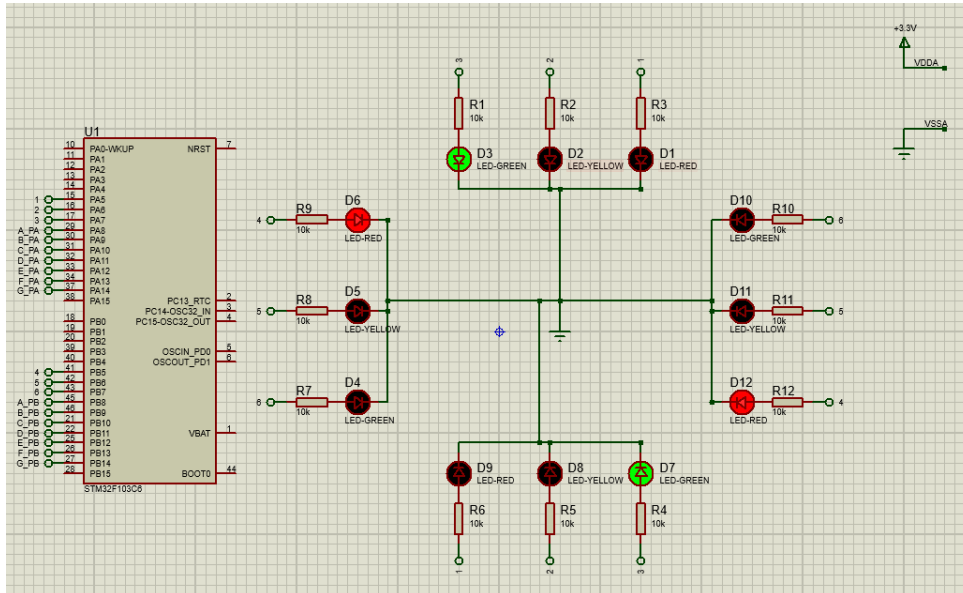
```
10 {
11     / USER CODE END WHILE /
12     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET    );
13     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET      );
14     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET      );
15     HAL_Delay(5000); // 5s delay for RED
16     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET      );
17     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET      );
18     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET    );
19     HAL_Delay(2000); // 2s delay for GREEN
20     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET      );
21     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET    );
22     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET      );
23     HAL_Delay(3000); // 3s delay for YELLOW
24     / USER CODE BEGIN 3 */
25 }
```

Listing 2: Code inside while loop

Since we are using an active low configuration, `GPIO_PIN_RESET` will be our logic 1 while `GPIO_PIN_SET` will be our logic 0. We then use this knowledge to configure the state for our "traffic lights" with 3 LEDs representing 3 states RED, YELLOW and GREEN with each separate LED.

3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light.



<https://github.com/kyoyamakang/MCU-MPU-Lab-Files-Archive>

```

1 void TrafficLight_4Way(void) {
2     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET ); // Traffic
3     Light A RED
4     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET );
5     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET );
6     // -----
7     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET ); //
8     Traffic light B green
9     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET );
10    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET );
11    HAL_Delay(2000);
12    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET ); //
13    Traffic light B yellow
14    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET );
15    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET );
16    HAL_Delay(3000); // 5s total delay for RED
17    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET ); // Trafifc
18    light B red

```

```
15 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET );
16 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET );
17 // -----
18 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET );
19 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET );
20 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET );
21 HAL_Delay(2000); // 2s delay for GREEN
22 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET );
23 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET );
24 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET );
25 HAL_Delay(3000); // 3s delay for YELLOW
26 };
```

Listing 3: User defined function

The logic for our 4 way traffic light is shown above with its core being split into 2 parts since the intersection is 2 roads crossed, we need only implement 2 lights and duplicate them to form our 4 way lights.

```
1 while (1)
2 {
3     TrafficLight_4Way();
4 }
```

Listing 4: Code inside while loop

Inside of the while loop, we call the function as defined above which runs our traffic light every instance of the while loop.

4 Exercise 4

Instead of the given configuration, we decided to rearrange some of the pins for the 7SEG LED display to be able to fit later exercises.

```
1 void display7SEG_PA(int num) {
2     switch(num) {
3         case 0:
4             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET ); // A
5             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET ); // B
6             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET ); // C
7             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET ); // D
8             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET ); // E
9             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET ); // F
10            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET ); // G
11            break;
12        case 1:
13            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET );
14            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET );
15            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET );
16            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET );
17            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET );
18            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET );
19            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET );
20            break;
21        case 2:
22            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET );
23            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET );
24            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_SET );
25            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET );
26            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET );
27            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET );
28            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET );
29            break;
30        case 3:
31            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET );
32            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET );
33            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET );
34            HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET );
```



```
35     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET    );
36     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET    );
37     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET  );
38     break;
39 case 4:
40     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET     );
41     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET   );
42     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET  );
43     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET    );
44     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET    );
45     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET  );
46     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET  );
47     break;
48 case 5:
49     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET   );
50     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET     );
51     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET  );
52     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET  );
53     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET    );
54     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET  );
55     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET  );
56     break;
57 case 6:
58     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET   );
59     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET     );
60     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET  );
61     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET  );
62     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET  );
63     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET  );
64     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET  );
65     break;
66 case 7:
67     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET   );
68     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET   );
69     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET  );
70     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_SET    );
71     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET    );
72     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_SET    );
```

```
73     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_SET    );
74     break;
75 case 8:
76     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET    );
77     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET    );
78     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET   );
79     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET   );
80     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET   );
81     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET   );
82     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET   );
83     break;
84 case 9:
85     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET    );
86     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET    );
87     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET   );
88     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET   );
89     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_SET     );
90     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET   );
91     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET   );
92     break;
93 default:
94     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET    );
95     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET    );
96     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_10, GPIO_PIN_RESET   );
97     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_11, GPIO_PIN_RESET   );
98     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_12, GPIO_PIN_RESET   );
99     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_13, GPIO_PIN_RESET   );
100    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_14, GPIO_PIN_RESET   );
101    break;
102 }
103 };
```

Listing 5: 7Segment LED display function - Active LOW

Using switch-case statement to handle integer input and output 8 separate signals driving the 7Segment LED to display numbers. If any invalid number is inputted, the default case is all segments will be lit up.



```
1 num = 4;
2 while (1)
3 {
4     /* USER CODE END WHILE */
5     display7SEG_PA(num);
6     /* USER CODE BEGIN 3 */
7 }
```

Listing 6: Code inside while loop

We define an arbitrary integer number and input it in our function.

5 Exercise 5

```
1 void TrafficLight_7LED_Display(int* state, int* countA, int*
   countB) {
2     switch (*state)
3     {
4         case 2:          // A RED - B GREEN
5             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
6             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
7             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
8
9             HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
10            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
11            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET);
12
13            display7SEG_PA(*countA);
14            display7SEG_PB(*countB);
15
16            (*countA)--;
17            (*countB)--;
18            if (*countB == 0)
19            {
20                *state = 3;
21                *countB = 3;
22            }
23            HAL_Delay(1000);
24            break;
25        case 3:          // A RED - B YELLOW
26            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_RESET);
27            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
28            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);
29
30            display7SEG_PA(*countA);
31            display7SEG_PB(*countB);
32
33            (*countA)--;
34            (*countB)--;
35            if (*countB == 0)
```



```
36     {
37         *state = 0;
38         *countA = 2;
39         *countB = 5;
40     }
41     HAL_Delay(1000);
42     break;
43     case 0:        // A GREEN - B RED
44         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
45         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
46         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_SET);
47
48         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, GPIO_PIN_SET);
49         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
50         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);
51
52         display7SEG_PA(*countA);
53         display7SEG_PB(*countB);
54
55         (*countA)--;
56         (*countB)--;
57         if (*countA == 0)
58         {
59             *state = 1;
60             *countA = 3;
61         }
62         HAL_Delay(1000);
63         break;
64     case 1:        // A YELLOW - B RED
65         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
66         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET);
67         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
68
69         display7SEG_PA(*countA);
70         display7SEG_PB(*countB);
71
72         (*countA)--;
73         (*countB)--;
```

```
74         if (*countA == 0)
75         {
76             *state = 2;
77             *countA = 5;
78             *countB = 2;
79         }
80         HAL_Delay(1000);
81         break;
82     }
83 };
```

Listing 7: User defined function

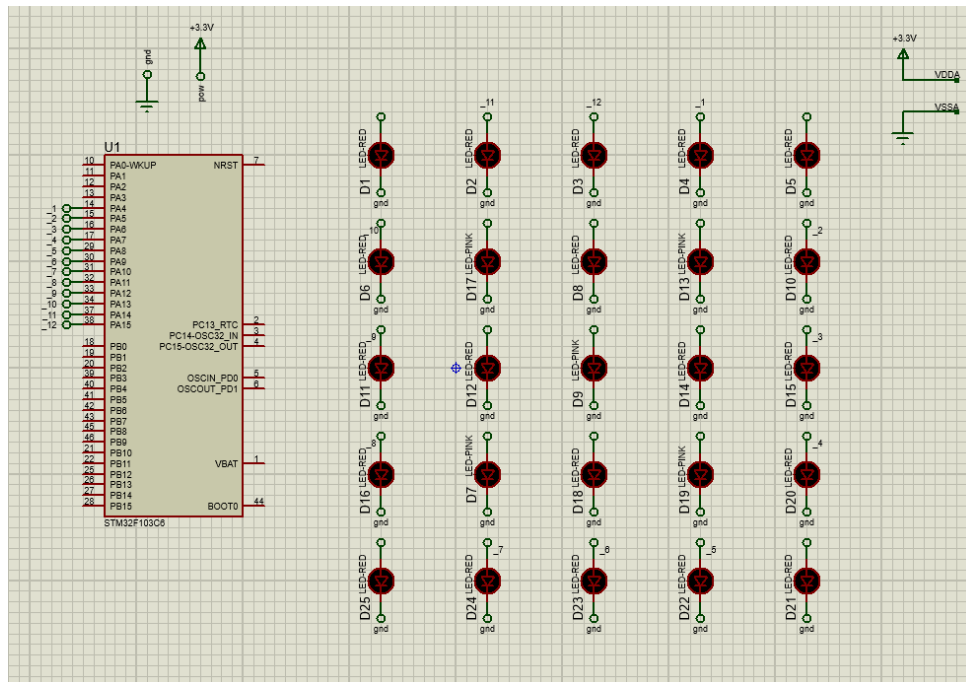
The function uses a stat parameter with 4 values, 2 counter flag A and B to control 2 different 7SEG LED display. The parameters are passed and are modified inside the function to update and control the functions of the components.

```
1  int state = 2;
2  int countA = 5;
3  int countB = 2;
4  while (1)
5  {
6      /* USER CODE END WHILE */
7      // TrafficLight_4Way();
8      TrafficLight_7LED_Display(&state, &countA, &countB);
9      /* USER CODE BEGIN 3 */
10 }
```

Listing 8: Code inside while loop

6 Exercise 6

In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32.



<https://github.com/kyoyamakhang/MCU-MPU-Lab-Files-Archive>

```
1 const uint16_t LED_PINS[12] = {
2     GPIO_PIN_15, // 12
3     GPIO_PIN_4,  // 1
4     GPIO_PIN_5,  // 2
5     GPIO_PIN_6,  // 3
6     GPIO_PIN_7,  // 4
7     GPIO_PIN_8,  // 5
8     GPIO_PIN_9,  // 6
9     GPIO_PIN_10, // 7
10    GPIO_PIN_11, // 8
11    GPIO_PIN_12, // 9
12    GPIO_PIN_13, // 10
13    GPIO_PIN_14  // 11
14 };
15
```



```
16 //Simple testing function
17 void clock(int* timer) {
18     HAL_GPIO_WritePin(GPIOA, LED_PINS[(*timer)], GPIO_PIN_SET);
19 };
```

Listing 9: User defined function and tester

LED pins are declared as a vector with constant value for easier and cleaner code.

```
1 int timer = 0;
2 while (1)
3 {
4     /* USER CODE END WHILE */
5     if(timer >= 12) timer = 0;
6     clock(&timer);
7     HAL_Delay(1000);
8     timer++;
9     /* USER CODE BEGIN 3 */
10 }
```

Listing 10: Testing

This simple tester function ensure the clock runs as a sequence correctly.

7 Exercise 7

```
1 void clearAllClock(void) {  
2     for(int i = 0 ; i < 12 ; i++) HAL_GPIO_WritePin(GPIOA , LED_PINS [  
3         i], GPIO_PIN_RESET);  
4 };
```

Listing 11: User defined function

The function reset all pins to their logic 0, turning all the LEDs off.

8 Exercise 8

```
1 void setNumberOnClock(int num) {  
2     clearAllClock();  
3     HAL_GPIO_WritePin(GPIOA , LED_PINS[num], GPIO_PIN_SET);  
4 };
```

Listing 12: User defined function

Using the previous `clearAllClock()` function, we can implement a set clock function presented above. The clearing is divided into 2 frames, one clear all lighting LEDs and the second turns on the designated LED.

9 Exercise 9

```
1 void clearNumberOnClock(int num) {  
2     HAL_GPIO_WritePin(GPIOA , LED_PINS[num], GPIO_PIN_RESET);  
3 };
```

Listing 13: User defined function

10 Exercise 10

```
1 void displayClock(int hour, int minute, int second) {  
2     clearAllClock();  
3  
4     int hour_pos = hour % 12;  
5     int min_pos  = minute / 5;  
6     int sec_pos  = second / 5;  
7  
8     HAL_GPIO_WritePin(GPIOA, LED_PINS[hour_pos], GPIO_PIN_SET);  
9     HAL_GPIO_WritePin(GPIOA, LED_PINS[min_pos],  GPIO_PIN_SET);  
10    HAL_GPIO_WritePin(GPIOA, LED_PINS[sec_pos],  GPIO_PIN_SET);  
11 };
```

Listing 14: User defined function

```
1 int hour = 0;  
2 int minute = 0;  
3 int second = 0;  
4 while (1)  
5 {  
6     displayClock(hour, minute, second);  
7     HAL_Delay(1000);  
8  
9     second++;  
10    if (second >= 60) {  
11        second = 0;  
12        minute++;  
13        if (minute >= 60) {  
14            minute = 0;  
15            hour++;  
16            if (hour >= 12) {  
17                hour = 0;  
18            }  
19        }  
20    }  
21 }
```

Listing 15: Testing



Since the clock has only 12 LEDs, the 60 minutes and seconds must be divided by 5 to have the proper representation. Hence the handles only moves once for every 5 minutes for the minute handle (or LED) and every 5 seconds for the second handle (or LED). If all 3 hands happen to be on the same place (LED), the LED can appear to blink for several times before stabilizing.

11 References

[1] Files can be found via the Github link

<https://github.com/kyoyamakhang/MCU-MPU-Lab-Files-Archive>